

MIS API Documentation for TAP School Payment System

This is a sample API documentation provided by MIS for integrating with the TAP School Payment System for a specific school.

1. Get Due Amount/ Due Check API

Retrieves the due amount for a specific reference ID and bill month.

- **Endpoint:** `https://{base_url}/fetch-due-amount`
- **Request Method:** `POST`
- **Request Headers:**
 - `Content-Type: application/json`
 - `client_id` : (string) MIS id for authentication.(OPTIONAL). Might be different for different school.
 - `client_secret` : (string) MIS key for authentication.(OPTIONAL). Might be different for different school.
- **Request Body Parameters:**
 - `student_id` : (string) Reference ID/student ID of the bill.
 - `bill_month` : (string) Bill month in the format MMYYYY.

Example Response:

```
{
  "code": 200,
  "message": "Success",
  "student_name": "Md. Mubin Rahman",
  "bill_month": "N/A",
  "bill_amount": 7025.00,
  "bill_due_date": "",
  "mis_tx_id": "MIS-SIDE-ID-ey6c2228-2cg1-5bh93973are2",
  "response_date_time": "2023-11-10 07:59:43"
}
```

Example Curl Request:

```
curl --location 'https://flash.mis.service.com/api/v1/TapPay/fetch-due-amount' \
--header 'Content-Type: application/json' \
--header 'client_id: 68eee04d1077483ghghhgagg' \
--header 'client_secret: 149541B02E23487EBE51770867567' \
--data-raw '{
    "student_id": "334",
    "bill_month": "122023"
}'
```

Due Check Error Messages:

1. Case 1: Institution Not Available

- code : 404
- message : "Invalid Institute"

2. Case 2: Bill Month Not Available

- code : 404
- message : "Invalid Bill Date"

3. Case 3: Student ID Not Found

- code : 404
- message : "Invalid ID"

4. Case 4: Combination of Student ID and Bill Month Not Found

- code : 404
- message : "Invalid Bill config"

5. Case 5: No Due

- code : 423
- message : "No Dues"

6. Case 6: Bill fetch success

- code : 200
- message : "Success"

2. Payment Collection Confirmation/ Payment API

Confirms the payment with the provided details.

- **Endpoint:** `https://{base_url}/process-payment`
- **Request Method:** `POST`
- **Request Headers:**
 - `Content-Type:` `application/json`
 - `client_id` : (string) MIS id for authentication.(OPTIONAL). Might be different for different school.
 - `client_secret` : (string) MIS key for authentication.(OPTIONAL). Might be different for different school.
- **Request Body Parameters:**
 - `student_id`: (string) Reference ID of the bill.
 - `bill_month`: (string) Bill month in the format MMYYYY.
 - `amount`: (float) Amount to be paid.
 - `user_mobile_Number`: (string) User's mobile number associated with the payment.
 - `tap_tx_id`: (string) TAP side transaction Id generated by TAP.
 - `pay_time`: (string) Date and time of the payment in the format YYYY-MM-DD HH:MM:SS.

Example Response:

```
{
  "code": 200,
  "message": "Success",
  "mis_tx_id": "MIS-SIDE-ID-ey6c2228-2cg1-5bh93973are2",
  "response_date_time": "2023-11-10 08:20:43"
}
```

Example Curl Request:

```
curl --location 'https://flash.mis.service.com/api/v1/TapPay/process-payment' \
--header 'Content-Type: application/json' \
--header 'client_id: 68eee04d1077483ghghhgggg' \
--header 'client_secret: 149541B02E23487EBE51770867567' \
--data-raw '{
    "student_id": "334",
    "bill_month": "122023",
    "amount": 7025.00,
    "user_mobile_Number": "01792726811",
    "tap_tx_id": "SPNWRUHCIX24240XRG",
    "pay_time": "2023-11-10 08:20:43"
}'
```

Payment Error Messages:

1. Case 1: payment success

- code : 200
- message : "Success"

2. Case 2: For any other response the payment is unsuccessful. Need to call inquiry API to know the status

- code : {not defined}
- message : {not defined}

3. Get Transaction Status/ Inquiry API

Retrieves the status of a specific transaction using its TAP Side transaction ID.

- **Endpoint:** `https://{base_url}/get-transaction-status`
- **Request Method:** POST
- **Request Headers:**
 - Content-Type: application/json
 - client_id : (string) MIS id for authentication.(OPTIONAL). Might be different for different school.
 - client_secret : (string) MIS key for authentication.(OPTIONAL). Might be different for different school.

- **Request Body Parameters:**

- `tap_tx_id`: (string) Transaction ID for which status is to be retrieved. TAP side transaction Id

Example Response:

```
{
  "code": 200,
  "message": "Success",
  "mis_tx_id": "MIS-SIDE-ID-iofc2225-5cs8-5bh93965are2",
  "student_id" : "334",
  "student_name": "Md. Mubin Rahman",
  "tran_date_time": "2023-11-10 08:20:43",
  "total_amount": 7025.00,
  "tran_status": "paid"
}
```

Example Curl Request:

```
curl --location 'https://flash.mis.service.com/api/v1/TapPay/get-transaction-status' \
--header 'Content-Type: application/json' \
--header 'client_id: 68eee04d1077483ghghhg' \
--header 'client_secret: 149541B02E23487EBE51770867567' \
--data-raw '{
  "tap_tx_id": "SPNWRUHCIX24240XRG"
}'
```

Inquiry Error Messages:

1. Case 1: Transaction Not Found

- `code` : 404
- `message` : “No transaction found with the core Id”

2. Case 2: Successful Transaction Found with the core Id

- `code` : 200
- `message` : “Success”

Notes:

- `https://flash.mis.service.com/api/v1/TapPay` is the example of `{base_url}`. It is not a working domain.
- When designing APIs, it is crucial to use HTTP status codes accurately to convey the result of a client's request. For example when designing APIs, avoid using HTTP 404 status codes to indicate that a requested resource, such as a customer record, does not exist. A 404 code implies that the endpoint is incorrect, which can lead to confusion. Instead, return an HTTP 200 OK status code with a descriptive response body, such as `{ "code": 404, "message": "Resource not found." }`. This approach clearly communicates that the request reached a valid endpoint while informing the client that the specific resource is missing.
- It's important to note that the response body can include custom codes or messages that do not necessarily adhere to standard HTTP codes, allowing for more specific error handling.
- Maintain `float` format for `bill_amount` and `Integer` format for 'code' for both request and response.
- While authentication through `client_id` and `client_secret` is strongly recommended, alternative methods are also supported
- Ensure that all requests and responses strictly adhere to the `application/json` format.
- Follow precise parameter formatting guidelines, especially for dates as documented.
- Implement comprehensive error handling to gracefully manage exceptions.